Quick Revision: Blockchain & Its Applications (NOC26_CS34)

Here's a focused revision covering all 12 assignments with key answers and concepts.

🔐 WEEK 1 — Cryptography Basics

- **SHA-256**: Fixed output, collision resistant, avalanche effect (tiny input change → completely different hash)
- **Collision attack** on 128-bit hash needs 2^{64} operations (birthday paradox: 2^N/2)
- **Avalanche effect**: Small input change flips nearly all output bits
- **Blockchain**: Distributed ledger, immutable, decentralized — NO single authority
- Moving to blockchain from centralized DB → **throughput decreases, control distributed**
- Block time 5 min, 50% hash rate for 4 hrs out of 12 → **120 blocks total**

🔑 WEEK 2 — Public Key Cryptography

- **RSA**: $\varphi(n) = (p-1)(q-1)$, find $d = e^{-1} \bmod \varphi(n)$
- **Encryption flow**: Alice encrypts with **Bob's public key** → Bob decrypts with **Bob's private key**
- **Signature flow**: Alice signs with **Alice's private key** → Bob verifies with **Alice's public key**
- **Non-repudiation**: Sender cannot deny authoring a signed transaction
- **Merkle Tree post-order** (8 leaves): leaves first, then internal nodes, root last → {1,2,12,3,4,34,1234,5,6,56,7,8,78,5678,12345678}
- **Nonce** is what miners modify to find valid block hash
- Bitcoin fork resolution → **Longest Chain Rule** (most cumulative PoW)
- Consensus solves → **Byzantine Generals Problem**

⛓️ WEEK 3 — Bitcoin Structure

- Block reward 3.125 BTC × 144 blocks/day = **450 BTC/day**
- **Permissioned blockchain**: Restricted validator set, access control layer
- Bitcoin block hash = **Double SHA-256** of current block header
- Header fields: Previous hash, timestamp, nonce, Merkle root, difficulty bits, version ✅

- Most difficult target = **most leading zeros**
- Merkle proof for 64 txns = $\log_2(64)$ = **6 hashing steps**
- Total nodes in Merkle tree (16 leaves) = 2n−1 = **31**
- Odd transactions in Merkle tree → **duplicate (dummy) hash added**
- Nonce is 32-bit → max $\mathbf{2^{32}}$ values (~4 billion)
- **OP_NIP** removes second-to-top stack item

🍴 WEEK 4 — Forks, Attacks, Consensus

- **Soft fork**: Backward-compatible upgrade
- **Hard fork**: Permanent chain split (e.g., Bitcoin Cash)
- Halving every 4 years from 2009 (100 BTC): 2013→50, 2017→25, 2021→12.5
- **Longest Chain Rule** prevents double-spending
- PoW prevents **Sybil attacks** (costly to create fake nodes)
- **Merkle Root** = fingerprint of all transactions in a block
- Bitcoin tolerates **Byzantine Faults** (not just crash faults)
- **Safety**: Transactions irreversible | **Liveness**: Eventually added
- **51% attack**: Can double-spend and censor — CANNOT steal funds or change supply cap
- Before mining: Must **construct candidate block** from mempool

---

⚡ WEEK 5 — Consensus Mechanisms & Ethereum

- Permissionless blockchain scalability limited by: **global agreement + high communication overhead**
- **PoET** (Proof of Elapsed Time): Uses Intel SGX trusted hardware for fair leader selection
- **PoS advantage over PoW**: Reduced energy consumption
- Fork resolution in PoW: **Heaviest chain** (most accumulated work)
- **EVM**: Ensures deterministic smart contract execution across all nodes
- State-modifying Ethereum functions require **gas + block inclusion**
- **Nonce** in Ethereum: Prevents replay attacks, ensures transaction ordering
- `eth_getBalance` → retrieves Ether balance
- **Proof of Burn (PoB)**: Destroy coins to prove commitment
- **1 Gwei = 10^9 Wei** ✅ (1 Ether = 10^{18} Wei = 10^9 Gwei)

🤝 WEEK 6 — Paxos & BFT

- **Safety**: No conflicting decisions | **Liveness**: System eventually progresses

- **Paxos** tolerates: **Crash faults** only (not Byzantine)
- BFT formula: **$n \geq 3f + 1$** → For n=7, max f=**2** Byzantine faults
- Quorums must intersect to **prevent two different values being chosen** (safety)
- Byzantine faults: **arbitrary/malicious behavior + conflicting messages to different nodes**
- Without majority in Paxos → **Liveness** affected (not safety)
- **Multi-Paxos**: Avoids repeated prepare phase by electing stable leader
- Paxos roles (Proposer/Acceptor/Learner): **dynamic, not permanently fixed**, one node can play all roles

---

🏢 WEEK 7 — PBFT & Hyperledger Fabric

- **PBFT suitable for enterprise** because: deterministic finality + Byzantine fault tolerance
- **MSP (Membership Service Provider)**: Defines identity validation rules and participant roles
- Byzantine quorum safety: **Any two quorums must share at least one honest replica**
- Blockchain in supply chain: **Tamper-evident audit trail + improved transparency**
- PBFT handles faulty primary via: **Timeout-based view change protocol**
- **Hyperledger Fabric Channels**: Logically separate ledgers for subsets of participants
- BFT $n \geq 3f+1$: Ensures two quorums of size $2f+1$ share **at least one honest replica**
- **Algorand committee**: Rotated every round, selected via **VRF (Verifiable Random Function)**
- Fabric key features: **Modular design + separation of endorsement and ordering**

🔗 WEEK 8 — Bitcoin-NG, ByzCoin, Fabric Flow

- **PBFT FALSE statement**: It is NOT effective for large decentralized public networks ($O(n^2)$ overhead)
- PBFT deterministic finality via: **Quorum-based agreement with overlapping honest replicas**
- **Bitcoin-NG**: Separates leader election (key blocks) from transactions (microblocks) → higher throughput
- Microblocks carry transactions, produced frequently; Key-blocks elect leader via PoW
- **CoSi improves**: Quadratic communication overhead of PBFT
- **ByzCoin**: Uses CoSi (collective signing) → aggregates signatures into one compact Schnorr signature
- ByzCoin benefit: **Reduces signature verification overhead**
- BFT vs PoW tradeoff: **Deterministic finality but limited scalability**
- Fabric endorsement before ordering: To **execute business logic + verify endorsement policy**
- Fabric validation: Checks **endorsement policies + read-write set consistency** before updating world state

---

🌐 WEEK 9 — ByzCoin, Algorand, DIDs, SSI

- **ByzCoin solves Bitcoin-NG limitation**: Faulty microblocks only discovered at end of round
- **Blockchain Performance Triangle**: Scalability, Security, Decentralization
- ByzCoin improvements: reduces latency to **O(log n)** + verification to **O(1)** via Schnorr multi-sigs
- **Algorand VRF sortition**: j=0 means **not part of the committee**
- **Weak synchrony**: Adversary can control network for **bounded period of time**
- **Single Sign-On FALSE**: Identity holders do NOT use IdP to authenticate other identity holders
- **DID URI syntax**: `did:<method-name>:<method-specific-identifier>`
- DID Document contains: public keys + authentication methods + **service endpoints**
- DID Documents can use JSON, XML etc. (NOT JSON only)
- **VC Issuer**: Creates and cryptographically signs claims about a subject
- Algorand BA*: Runs **2-3 times** typically sufficient; uses timeout + new random committee
- In `did:example:123` → 'example' = **the DID method**

🔀 WEEK 10 — Cross-Chain, HTLCs, Interoperability

- **HTLC**: Funds locked until secret revealed OR timeout → sender gets refund
- Atomic swap halt after secret revealed → conforming party **either gets asset or retains original**
- Centralized TTP risks: **Relinquish control + single point of failure + lose funds if compromised**
- **Consensus-driven data transfer** (permissioned): Ensures data is **finalized, not temporary/unconfirmed**
- Three-party swap timelocks **(6Δ, 5Δ, 4Δ)**: Staggered to give each party buffer to react
- Permissioned interoperability data must include: **Endorsements/signatures + Exposure Control compliance**
- Hyperledger Indy roles: **Stewards** write to ledger + approve Trust Anchors; **Trust Anchors** issue credentials
- **Data Acceptance Policy**: Evaluates if sufficient valid attestations from source network received
- **Genesis Transaction file**: Provides IP addresses, ports, keys of Indy pool nodes
- `support_revocation: False` → University **cannot revoke that credential** in future

---

🛡️ WEEK 11 — Aries, Attacks, Privacy

- **Hyperledger Aries fills**: Peer-to-peer interaction infrastructure gap (transport layer for credentials)
- ACA-Py components: **Core Capabilities + REST API + Aries Protocols (DIDcomm)**
- Selfish mining (2 blocks @ 6.25 BTC each) = **12.5 BTC reward**
- **Chaff coins/Mixins**: Used as decoys to prevent transaction privacy leakage

- Eclipse attack: Attacker controls **40% vs victim 30%** and **40% vs rest 30%** separately (honest 60% can't combine)
- Fomo3D suppression: 3 min = 180s ÷ 30s/ticket = **6 purchases prevented**
- **Eclipse Attack goal**: Isolate target by filling peer connections with attacker-controlled nodes
- Defense against eclipse: **Redesign IP bucket management** so honest IPs replace malicious ones
- **Double-spending with mining pool**: Attacker gets goods, pool confirms conflicting tx returning coins
- Eclipse isolation mechanism: **Filling victim's peer tables with attacker-controlled IPs**

🌍 WEEK 12 — Real-World Applications

- Best blockchain use case: **Transparent, tamper-evident supply chain tracking**
- **Project Ubin Phase 2**: Solves gridlock (banks lack liquidity for individual payments but are net-positive)
- **SCP (Stellar) key steps**: Vote → Accept → Confirm
- Ripple UNL overlap recommended: **Over 90%**
- CollabFed verification: **SPV + 2/3 supermajority approval** from service providers
- Blockchain access log for government data: **Minimizes breach risk + transparent access record**
- CollabFed signature aggregation: **CoSi with BLS signatures**
- **Project Ubin Phase 3 (DvP)**: Atomic exchange of cash and securities across systems
- Stellar performance: **~350 TPS + ~1 second finality**
- Decentralized marketplace challenge: **Lack of quick authority for real-time dispute resolution**

🎯 High-Frequency Answer Patterns

Most questions had these as correct answers — watch for them:

Topic	Watch Out For
BFT formula	$n \geq 3f+1$, quorum = $2f+1$
Merkle tree	$2n-1$ nodes, $\log_2 n$ proof steps
Bitcoin	Double SHA-256, Nonce modified, Longest chain
PBFT	$O(n^2)$ overhead, NOT for large networks
Ethereum	EVM = deterministic, Nonce = replay prevention
Attacks	Eclipse = fill peer tables, 51% = double spend only
DIDs	did:method:identifier, DID Doc has keys + endpoints

Good luck on your exam tomorrow! 🍀 Focus especially on Weeks 6-12 as those are the more recent and often tricky ones.